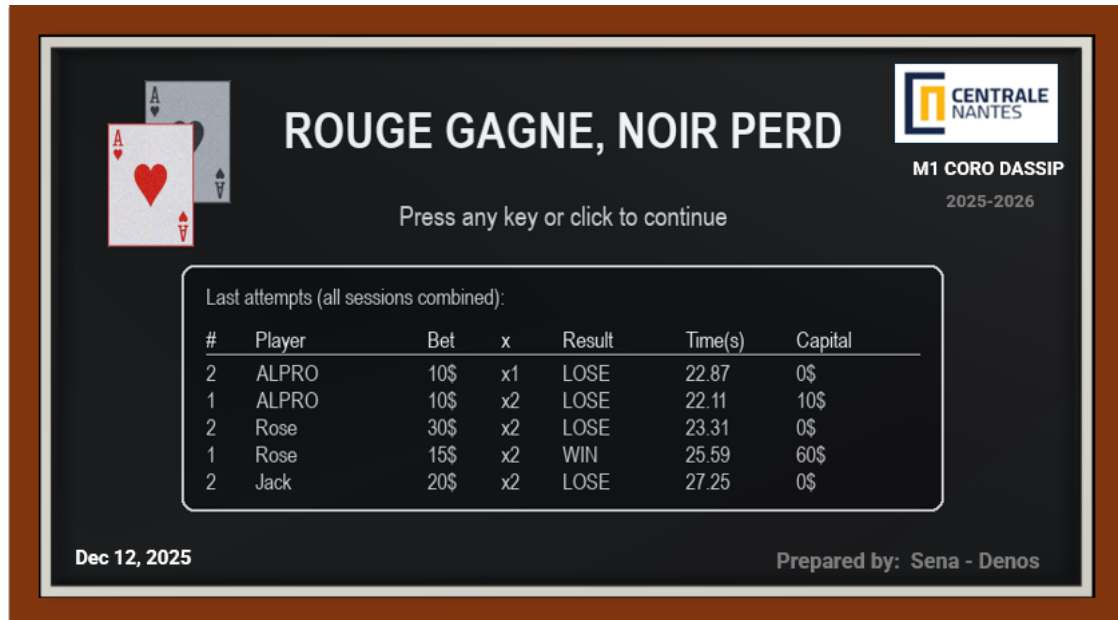


SOFTWARE ENGINEERING PROJECT REPORT

Rouge Gagne, Noir Perd

Interactive Card-Tracking and Betting Game in Python/Pygame



Participants

Denos Kume · Sena FUKABE

Instructor

Mira Rizkallah

Course

ALPRO - Algorithmics and Programming

Academic Year

2025-2026

Original project: December 2025 / Revised repository edition: September 2026

Abstract

Purpose, implementation approach, validation and main engineering outcomes

Rouge Gagne, Noir Perd is a single-player card-tracking game developed in Python with Pygame. The player observes one red card among two black cards, follows the cards through an animated shuffle, and selects the card believed to be red. The project was designed as a compact software-engineering exercise combining object-oriented programming, event-driven execution, finite-state control, 2D graphics, audio feedback, betting rules, input validation and persistent round history.

The final implementation separates player data, betting rules, game-state control and rendering into dedicated modules. A CardGame controller coordinates nine explicit runtime states, including pause/resume behavior, while JSON persistence records up to 200 recent rounds. Functional validation covered nickname rules, betting bounds, turbo stake capping, timed phase transitions, shuffle consistency, win/loss resolution, timeout handling, game-over behavior and history persistence. The repository edition was additionally validated under WSL Ubuntu 24.04 with Python 3.12.3 and Pygame 2.6.1; WSLg audio support was verified through PulseAudio client libraries.

The project demonstrates how a small interactive game can be structured as a maintainable software system rather than a monolithic script. The main remaining limitations are the absence of an automated unit-test suite, a fixed-size desktop interface and local-only single-player execution.

Acknowledgements

We thank Mira Rizkallah for her guidance throughout the ALPRO course and for connecting algorithmic concepts with practical software design. We also appreciate the collaborative effort involved in designing, implementing, testing, and documenting this project, as well as the constructive classroom environment that supported its iterative development.

Project at a Glance

Item	Implementation
Language / framework	Python 3.12 · Pygame 2.6.1
Core paradigm	Object-oriented + event-driven programming
Runtime control	Finite-state machine with pause/resume
Game window	960 × 630 pixels; main loop capped at 60 FPS
Persistence	Local JSON history; maximum 200 recent rounds
Validated platforms	Original Windows workflow; final repository additionally validated on WSL Ubuntu 24.04

Contents

Major sections of the revised report

1. Introduction	4
2. Requirements Engineering	5
3. System Architecture and Design	7
4. Implementation	10
5. User Interface and Gameplay	12
6. Verification and Testing	15
7. Results and Discussion	17
8. Risk Analysis and Future Work	18
9. Conclusion	19
References	20
Appendix A. Reproduction Guide	21

Section numbering and page references correspond to the PDF edition generated with this report.

1. Introduction

Context, game rules, project scope and engineering objectives

1.1 Context and Motivation

The project was developed for the ALPRO module as a practical application of programming and software-design concepts. Rather than implementing a purely console-based exercise, the team selected an interactive card game to combine algorithms with a graphical interface, timing constraints, user input, state transitions and persistent data.

The resulting system is intentionally small enough to understand end-to-end, but rich enough to require clear boundaries between domain logic, rendering, input handling and runtime state. This makes it suitable as an educational example of how object-oriented design and event-driven programming interact in a real application.

1.2 Game Concept and Rules

Each round uses three cards: one red card and two black cards. The player proceeds through three timed phases:

1. Observation - the colored card backs remain visible for 10 seconds so the player can identify the red card.
2. Shuffle - the cards are shown with identical fronts while random pairs swap positions for 10 seconds.
3. Selection - the player has 10 seconds to click the card believed to be red. A timeout counts as a loss.

The player begins with a virtual balance of \$30. A base bet is selected between \$10 and \$100 in \$5 increments, together with a turbo multiplier of $\times 1$, $\times 2$ or $\times 3$. The configured stake is the base bet multiplied by the turbo factor; the effective stake is capped by the current balance. A correct selection adds the effective stake as profit, while a wrong selection or timeout deducts it. The game ends when the balance falls below the minimum bet.

```
configured_stake = bet_amount * turbo
effective_stake = min(configured_stake, current_balance)
```

```
WIN: new_balance = current_balance + effective_stake
LOSS: new_balance = current_balance - effective_stake
```

1.3 Objectives and Scope

- Implement a complete playable loop with clear visual feedback and deterministic state transitions.
- Apply object-oriented decomposition to player, betting, card and controller responsibilities.
- Use timers and animation without blocking the Pygame event loop.
- Persist meaningful round data locally for later inspection.
- Demonstrate requirements, architecture, validation and risk analysis in an academic software-engineering report.

The scope is deliberately limited to a single-player, offline desktop application. The balance is virtual and the project does not connect to payment, gambling or online services.

2. Requirements Engineering

Traceable functional requirements, quality attributes and constraints

2.1 Functional Requirements

ID	Requirement	Acceptance condition
FR-01	Player profile	The system shall accept a nickname of 3-20 characters and allow the user to select one of three avatars.
FR-02	Bet configuration	The player shall adjust the base bet between \$10 and \$100 in \$5 increments.
FR-03	Turbo multiplier	The player shall select a multiplier of $\times 1$, $\times 2$ or $\times 3$; the effective stake shall never exceed the current balance.
FR-04	Round initialization	Each round shall create three cards and randomly assign exactly one red card.
FR-05	Observation phase	Card colors shall remain visible for 10 seconds before the shuffle begins.
FR-06	Shuffle phase	The system shall animate repeated random pair swaps for approximately 10 seconds while preserving card identity.
FR-07	Selection phase	The player shall have 10 seconds to select a card; no selection shall be treated as a timeout loss.
FR-08	Round resolution	The system shall update the balance according to the effective stake and display a win/loss result.
FR-09	Game over	The system shall enter GAME_OVER when the balance becomes lower than the minimum allowed bet.
FR-10	History persistence	Every completed round shall be appended to local JSON history with result, stake, duration and balance information.
FR-11	Pause / resume	SPACE shall pause supported active states and resume them without corrupting timers.
FR-12	Audio feedback	Shuffle, win and loss sounds shall play when audio is available; unavailable audio shall not prevent gameplay.

2.2 Non-Functional Requirements

ID	Quality attribute	Requirement
NFR-01	Responsiveness	The main loop is capped at 60 FPS and must remain event-responsive during timed phases.
NFR-02	Usability	The workflow shall expose clear labels, timers, buttons and visual state changes without requiring command-line interaction.
NFR-03	Maintainability	Domain responsibilities shall be separated across focused modules rather than implemented in one script.

ID	Quality attribute	Requirement
NFR-04	Robustness	Invalid input, missing assets/history and unavailable audio shall be handled without terminating the core game loop where practical.
NFR-05	Portability	The code shall run on supported desktop Python/Pygame environments; the repository edition was verified on WSL Ubuntu in addition to the original Windows workflow.
NFR-06	Data integrity	Round history shall remain valid JSON and shall be capped to avoid unbounded growth.

2.3 Constraints and Assumptions

- Single-player and offline execution only.
- Fixed 960 × 630 desktop window.
- Mouse-driven interaction for buttons and card selection; keyboard used for nickname entry and pause/resume.
- Local JSON file used instead of an external database.
- Random shuffle is intended for gameplay, not cryptographic security.
- Audio depends on an SDL-compatible device/backend; gameplay remains functional when audio is unavailable.

3. System Architecture and Design

Modules, runtime states, round flow and persistence model

3.1 Module Architecture

The final repository uses five Python modules. `main.py` owns application startup and the Pygame loop. `User` and `Bet` are small domain objects. `CardGame` is the controller that owns runtime state, cards, timers, round resolution and persistence. `dashboard.py` contains state-specific rendering functions. This separation keeps drawing code out of the game-state controller and avoids placing betting or balance rules inside the UI layer.

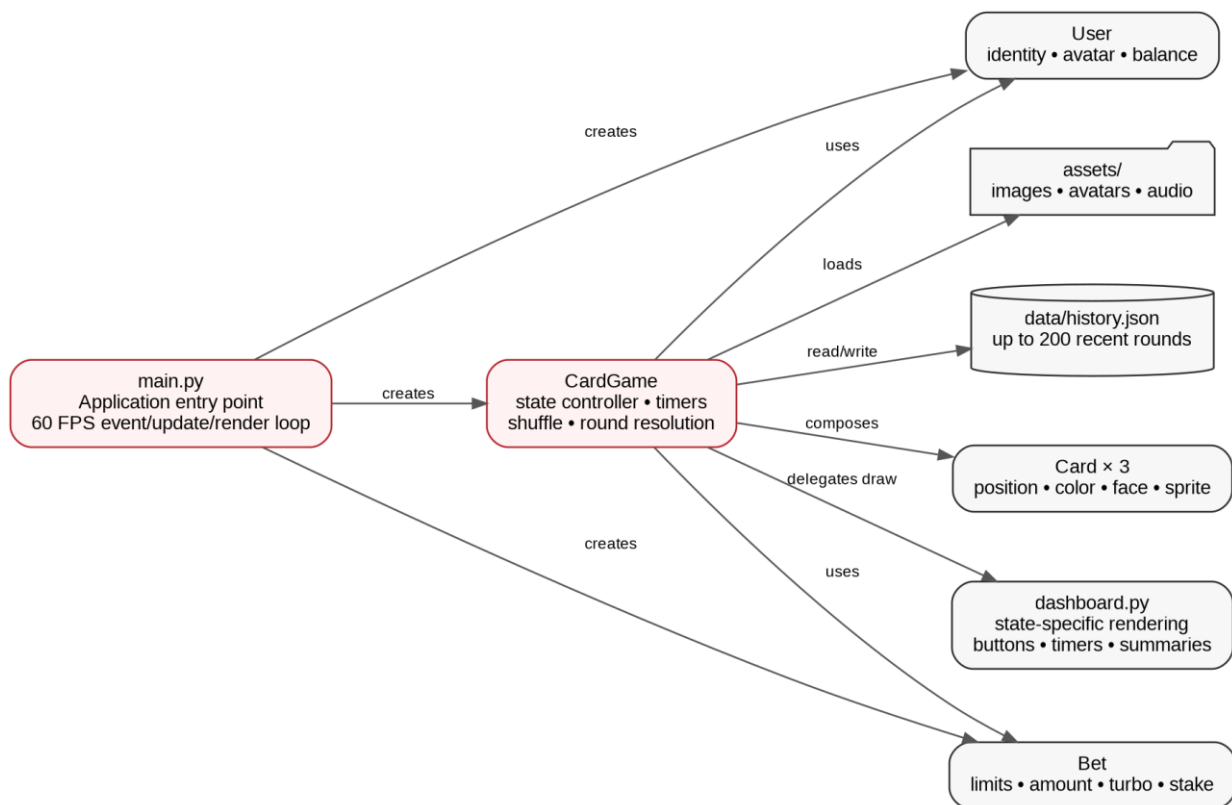


Figure 1. Module-level architecture of the final CardGame implementation.

3.2 Module Responsibilities

Module	Responsibility
main.py	Initialize Pygame, configure the window and audio, instantiate domain objects, and run the 60 FPS event/update/draw loop.
user.py	Maintain nickname, avatar index, initial balance and current balance; apply wins/losses and reset the player.
bet.py	Maintain minimum/maximum bet, current amount and turbo multiplier; enforce bounds and compute the configured stake.
game.py	Implement Card and CardGame, state transitions, timers, shuffle animation, input dispatch, balance resolution, pause handling and JSON persistence.

Module	Responsibility
dashboard.py	Render start/menu/bet/round/result/game-over screens, controls, timers, player information and summaries.

3.3 Finite-State Control

CardGame uses named states rather than deeply nested procedural logic. Event handling, time-based updates and rendering are dispatched according to the current state. This makes illegal transitions easier to prevent and keeps phase-specific behavior localized.

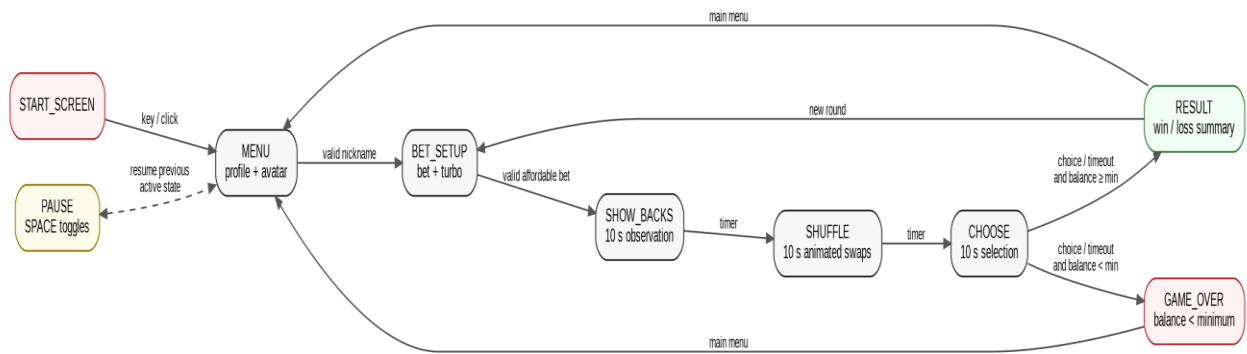


Figure 2. Runtime state machine. SPACE can temporarily enter PAUSE from most active states and then resume the previous state.

3.4 Round Processing Flow

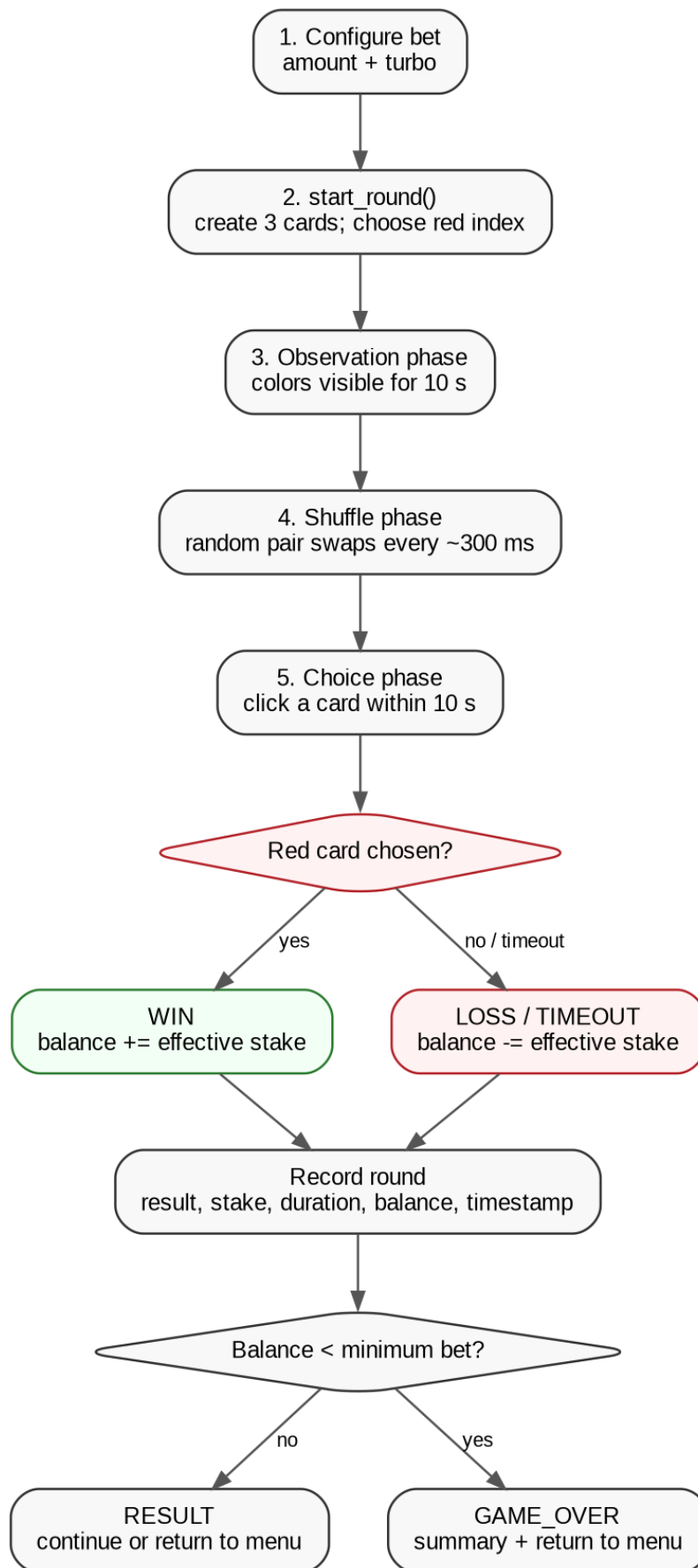


Figure 3. Round lifecycle from bet configuration to result or game over.

3.5 Persistence Model

Persistence is intentionally lightweight. The controller stores each completed round as a dictionary and serializes the global list to data/history.json. The file is created at runtime and excluded from Git. To prevent unbounded growth, only the 200 most recent entries are retained.

Field	Meaning
round	Round number within the current session
player	Trimmed nickname
bet	Base bet amount
mult	Turbo multiplier
stake	Effective stake used for balance update
result	WIN or LOSE
chosen_index	Selected card index, or null on timeout
red_index	Actual red-card index
duration	Round duration in seconds
balance_after	Player balance after resolution
timestamp	Unix timestamp for the completed round

4. Implementation

Core classes, timing, animation, UI rendering, audio and portability

4.1 Development Environment

The project was originally developed and documented in a Windows desktop workflow. The repository edition was later restructured and validated under WSL Ubuntu 24.04 using VS Code, Python 3.12.3 and Pygame 2.6.1 (SDL 2.28.4). Pygame is the only direct runtime dependency listed in requirements.txt; standard-library modules provide randomness, timing, file paths and JSON serialization.

Component	Validated repository environment
Python	3.12.3
Pygame	2.6.1 (SDL 2.28.4)
OS validation	WSL Ubuntu 24.04; original Windows workflow preserved
Editor	VS Code
Audio under WSLg	PulseAudio client library via libpulse0; pulseaudio-utils used for diagnostics

Component	Validated repository environment
Window / loop	960 × 630 pixels; 60 FPS cap

4.2 User and Bet Domain Objects

User encapsulates identity and balance operations. `apply_win(stake)` adds the net profit to the balance, while `apply_loss(stake)` deducts the effective stake. Bet encapsulates the allowed range, \$5 adjustment step and turbo multiplier. This keeps monetary rules outside the UI rendering functions.

A subtle but important rule is that the turbo multiplier can produce a configured stake larger than the current balance. The round resolver therefore caps the effective stake at the available balance before applying a win or loss. This behavior is covered by the functional test plan.

4.3 Card Representation and Shuffle Animation

Each Card stores a Pygame rectangle, a red/black identity and the current face state. At round creation, the red card index is selected randomly. During the shuffle phase, random pairs are selected with `random.sample(range(3), 2)`. Their rectangles are interpolated over approximately 300 ms using linear interpolation, producing visible movement while preserving the identity of the card objects.

Because the red/black property remains attached to the Card object while only rectangles exchange positions, the logical target cannot be lost during an animation. This design directly supports the core tracking mechanic.

4.4 Timers and Event-Driven Execution

The implementation avoids blocking sleep calls. Phase durations are measured with `pygame.time.get_ticks()`, allowing the event loop to remain responsive while countdowns and animations progress. `SHOW_BACKS`, `SHUFFLE` and `CHOOSE` each use a 10-second phase timer. Pause/resume shifts stored timestamps by the pause duration so that active timers do not expire while the game is paused.

4.5 Rendering Layer

`dashboard.py` contains the visual functions for each state, including profile creation, betting controls, round timers, player information, result panels and game-over summaries. `CardGame` decides what state is active; `dashboard.py` decides how that state is drawn. This controller/view split reduces visual clutter in the game logic.

4.6 Audio and Graceful Degradation

Three MP3 assets provide shuffle, win and loss feedback. Audio is treated as optional: sounds and channels are created only when `pygame.mixer` is successfully initialized. If the audio backend is unavailable, the game prints a diagnostic message and continues silently rather than terminating.

During final WSL validation, SDL initially failed because the PulseAudio client library `libpulse-simple.so.0` was unavailable. Installing `libpulse0` and `pulseaudio-utils` restored sound through the WSLg PulseAudio server. The repository entry point also supports selecting the WSLg PulseAudio backend before Pygame initialization. This debugging outcome improved portability and made audio failure non-fatal.

5. User Interface and Gameplay

Screens, interaction flow and feedback during a complete session

5.1 Player Profile and Betting

The profile screen requires a nickname before continuing and offers three avatar choices. The betting screen then exposes the current balance, base bet, turbo multiplier and controls for starting the round. Invalid or unaffordable configurations do not start a round.

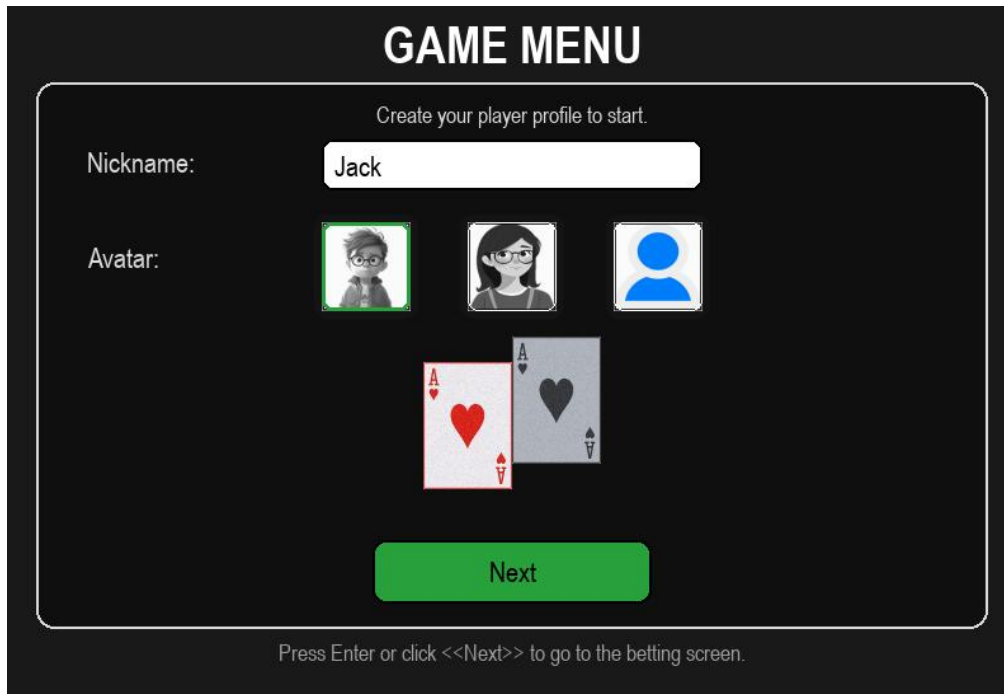


Figure 4. Player profile screen with nickname validation and avatar selection.

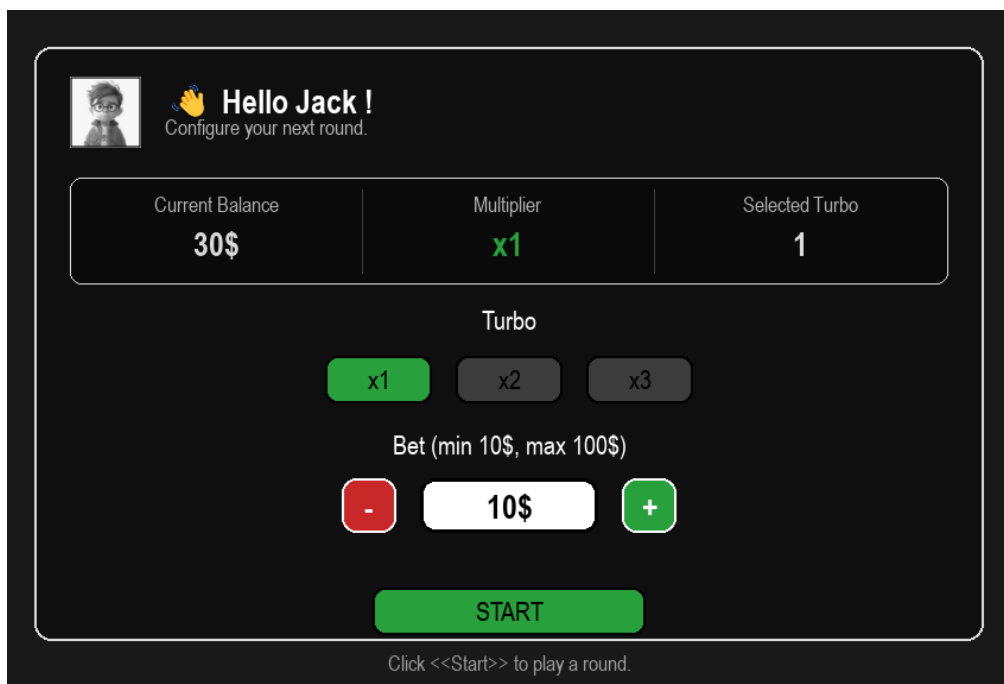


Figure 5. Betting dashboard showing balance, base bet and turbo controls.

5.2 Timed Round Phases

The round interface deliberately changes visual information between phases. During observation the player can identify the red card. During shuffle, the cards use the same visible front so success depends on tracking motion rather than color. During selection, the player must commit to one card before the countdown expires.

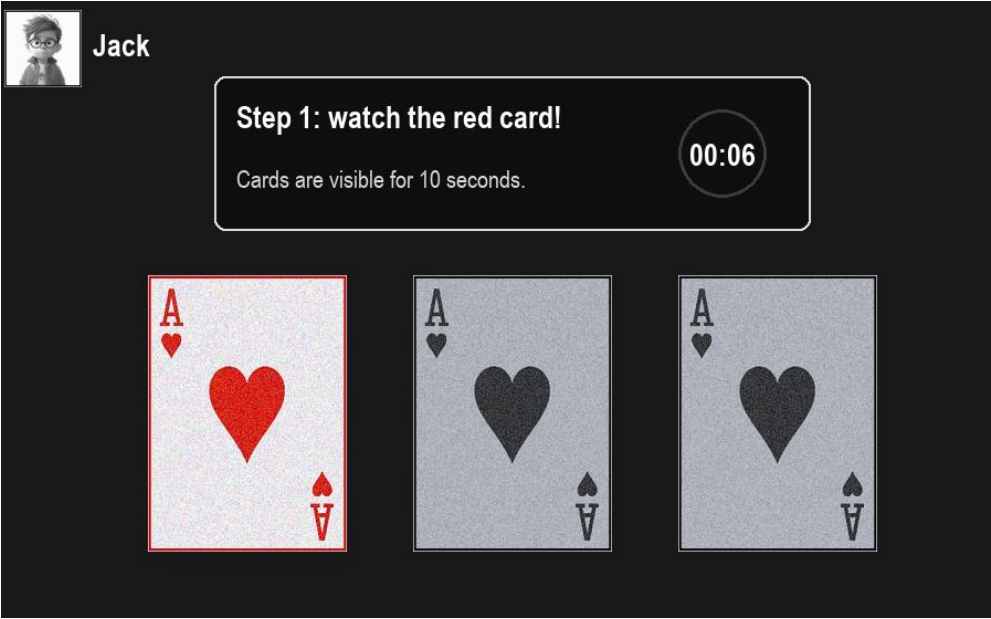


Figure 6. Observation phase: one red card and two black cards are visible.

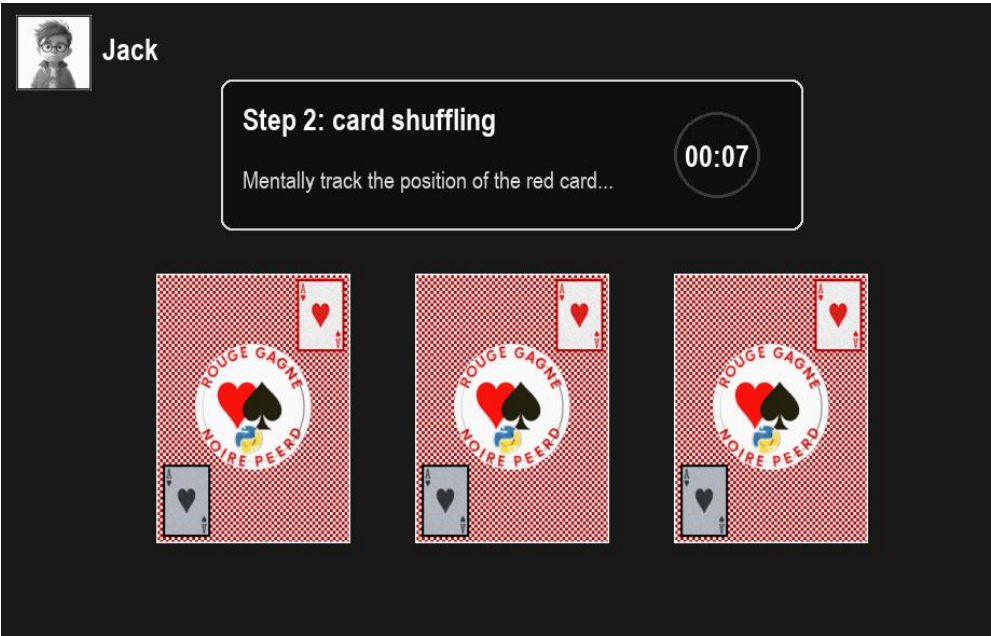


Figure 7. Shuffle phase: identical fronts move through animated swaps.

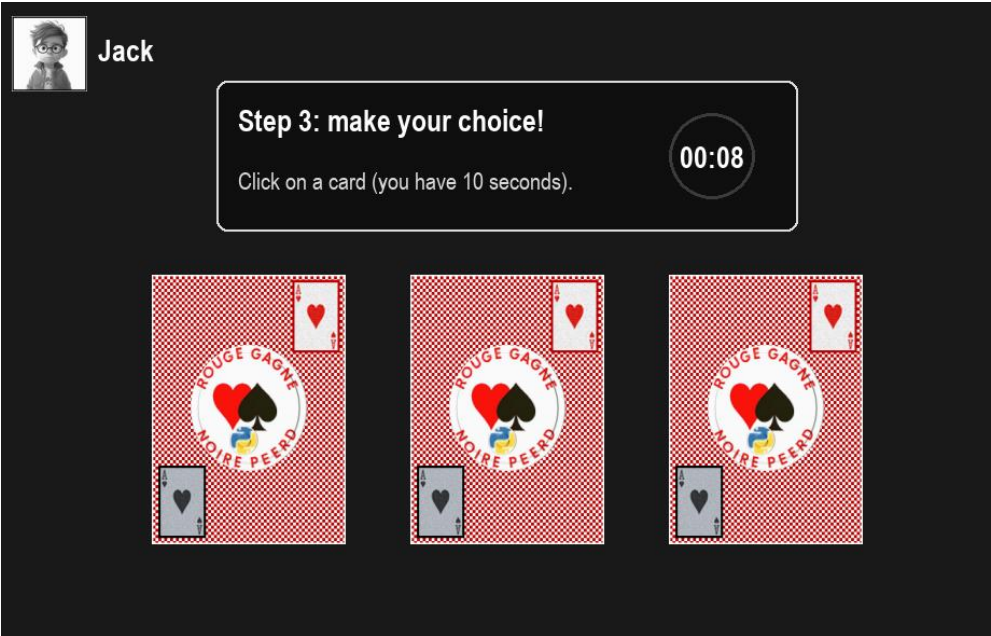


Figure 8. Choice phase: the player selects the tracked card within the time limit.

5.3 Results, History and Game Over

After selection or timeout, the result screen reports the outcome, configured bet, multiplier, gain or loss, new balance and round duration. If the balance falls below the minimum bet, the GAME_OVER state replaces the normal result continuation and presents a session summary. Recent global history is also visible on the start screen.

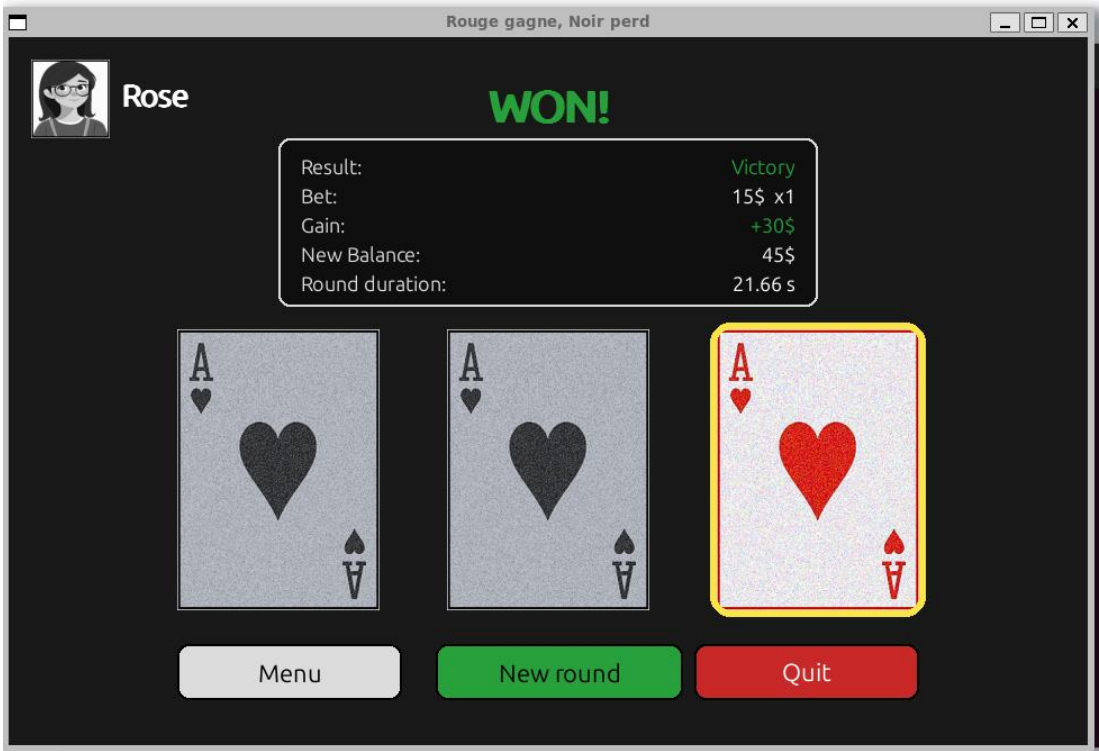


Figure 9. Verified runtime example showing a successful round in the final WSL repository environment.



Figure 10. Game-over summary after the balance becomes insufficient for a new round.

6. Verification and Testing

Functional traceability, runtime validation and quality checks

6.1 Test Strategy

The original project used a manual functional test plan focused on externally observable behavior. The revised report preserves those results and adds repository-level validation performed during final packaging. There is currently no automated unit-test suite, so the report distinguishes manual verification from design targets rather than presenting all properties as measured benchmarks.

6.2 Functional Test Results

ID	Feature	Test	Expected result	Status
TC01	Nickname	Less than 3 characters	Continue button disabled	Pass
TC02	Bet limit	Bet outside 10-100 range	Adjusted/prevented by bounds	Pass
TC03	Turbo	High bet with ×3 multiplier	Effective stake capped by balance	Pass
TC04	Card display	10-second observation duration	Transition to shuffle	Pass
TC05	Shuffle	Repeated card-position swaps	Red-card identity preserved	Pass
TC06	Win condition	Click the red card	Balance increases by effective stake	Pass
TC07	Loss condition	Click a black card	Balance decreases by effective stake	Pass
TC08	Timeout	No click within 10 seconds	Automatic loss	Pass
TC09	Game over	Balance falls below \$10	GAME_OVER screen displayed	Pass
TC10	History persistence	Complete a round	Entry appended to JSON history	Pass

6.3 Repository Runtime Validation

Check	Method	Observed / expected outcome	Result
Dependency integrity	python -m pip check	No broken requirements found	Pass
Application startup	python src/main.py	Pygame window opens and game loop runs	Pass
Round execution	Profile → bet → observe → shuffle → choose → result	Complete interactive round observed	Pass
Audio initialization	WSLg + PulseAudio client libraries	Shuffle/win/loss audio operational	Pass
Audio fallback	Mixer unavailable path	Game remains playable without crashing	Implemented
Persistence path	Runtime data/history.json	History created outside source tree and excluded from Git	Pass

6.4 Non-Functional Evaluation

Aspect	Evidence	Assessment
Performance	Main loop uses clock.tick(60).	60 FPS is a design cap; no hardware-wide benchmark is claimed.
Usability	State-specific labels, timers and on-screen buttons guide the round.	Interface supports the three-step gameplay flow without command-line input.
Robustness	Nickname/bet boundaries, guarded JSON I/O and audio fallback.	Known invalid-input and audio-device failures are handled without breaking core gameplay.
Persistence	Repeated rounds append structured records and the history list is capped.	JSON remains bounded to the 200 most recent entries.
Portability	Original Windows environment plus WSL Ubuntu validation.	Core game is portable; audio backend setup may be environment-specific.

7. Results and Discussion

What the implementation demonstrates, trade-offs and engineering lessons

7.1 Engineering Outcomes

- A complete event-driven desktop game was implemented without blocking the main loop.
- Explicit states make the user journey and timer-driven transitions easy to inspect and extend.
- Domain logic is separated from screen rendering, improving maintainability compared with a single-script design.
- The shuffle preserves card identity while animating position, directly supporting the game mechanic.
- Round history provides lightweight persistence without introducing a database dependency.
- Optional audio and asset fallbacks reduce environment-specific failure impact.

7.2 Design Trade-offs

The architecture favors clarity over abstraction. CardGame remains the largest component because it coordinates state transitions, timers, animation and persistence. For a project of this size, this keeps control flow visible; for a larger game, these responsibilities would likely be split into dedicated state objects, services or scene classes.

JSON is appropriate for a local teaching project because it is transparent and easy to inspect. It is not suitable for concurrent users, large-scale analytics or transactional guarantees. Similarly, the fixed desktop resolution simplifies layout but limits accessibility and responsiveness across different displays.

7.3 Lessons from Final Repository Validation

The WSL audio issue illustrates an important software-engineering lesson: a feature can be logically correct while still failing because of its runtime environment. The initial mixer failure was traced to a missing PulseAudio client library, not to the sound files or game rules. Treating audio as optional prevented the failure from taking down the entire application, while backend-specific diagnostics allowed sound to be restored later.

The same principle applies to persistence and graphical assets: external resources should fail gracefully whenever the core interaction can continue. This turns a classroom prototype into a more resilient repository artifact.

7.4 Current Limitations

- No automated unit or integration test suite; current verification is mainly manual.
- Single-player, offline execution only.
- Fixed-size desktop interface; no responsive layout or mobile support.
- Random shuffle is gameplay-oriented and not intended for cryptographic fairness guarantees.
- Rendering and controller logic are separated by module, but CardGame still carries several responsibilities.
- Cross-platform audio may require environment-specific SDL/PulseAudio configuration.

8. Risk Analysis and Future Work

Operational risks, mitigations and prioritized extensions

8.1 Risk Register

Risk	Likelihood	Impact	Mitigation
Invalid player input	Medium	Medium	Validate nickname length; clamp bet and turbo values; prevent invalid round start.
Audio backend unavailable	Medium	Low	Initialize mixer defensively, continue silently on failure, document WSLg PulseAudio dependency.
History file corrupt/unwritable	Low	Medium	Guard JSON load/save with exception handling; default to an empty list on read failure.
Low-end performance / animation timing	Medium	Low	Cap loop at 60 FPS, keep only three moving cards, use short linear interpolation.
Missing image/audio assets	Low	Low	Use guarded asset loading and visual fallback rectangles where implemented.
Controller growth	Medium	Medium	Refactor into state/scene classes if new modes significantly expand behavior.

8.2 Future Work

Extension	Engineering direction
Automated testing	Extract pure game-rule functions and add pytest tests for betting, resolution, state transitions and history serialization.
Difficulty modes	Vary shuffle interval, swap duration, observation time and card count.
Responsive UI	Scale layout to window size and improve keyboard accessibility.
Web/mobile port	Reimplement the presentation layer for a browser or mobile runtime while preserving core rules.
Analytics	Use history data for win rate, response time, stake distribution and session summaries.
Multiplayer / leaderboard	Introduce authenticated users and server-side persistence if the project evolves beyond local play.
Internationalization	Separate UI strings to support English/French language packs.

9. Conclusion

Final assessment of the project

Rouge Gagne, Noir Perd meets its original objective of turning core programming concepts into a complete interactive application. The project combines object-oriented entities, an explicit state machine, non-

blocking timers, animated card movement, input validation, balance rules, JSON persistence and audio-visual feedback in a coherent desktop game.

The revised repository and report strengthen the original submission in two ways. First, the documentation now reflects the actual implementation rather than relying on generic or partially outdated diagrams. Second, final runtime validation exposed and resolved a real portability issue in the audio stack while preserving a graceful silent fallback. This demonstrates the value of validating software in the environment in which it will actually be reused.

The main opportunity for further improvement is automated testing and additional separation of controller responsibilities. Even without those extensions, the current project provides a clear and reproducible example of event-driven Python game development and software-engineering practice at M1 level.

Repository Availability

Project path: github.com/denoskume/Master_SIP_EC-Nantes/tree/main/Projects/CardGame

The repository includes the source code, assets, README, academic documentation, gameplay GIF and MP4 demo.

References

Documentation and course sources used by the project

- [1] Python Software Foundation, "Python 3 Documentation," <https://docs.python.org/3/>.
- [2] Pygame Community, "Pygame Documentation," <https://www.pygame.org/docs/>.
- [3] Python Software Foundation, "json - JSON encoder and decoder," <https://docs.python.org/3/library/json.html>.
- [4] Centrale Nantes, ALPRO course materials, M1 CORO DASSIP, 2025.

Appendix A. Reproduction Guide

Project structure, installation and execution

A.1 Project Structure

```
CardGame/
├── src/
│   ├── assets/
│   ├── bet.py
│   ├── dashboard.py
│   ├── game.py
│   ├── main.py
│   └── user.py
├── data/
│   └── .gitkeep
├── docs/
│   ├── Report.pdf
│   ├── Game_Presentation.pptx
│   ├── cardgame_demo.gif
│   └── cardgame_demo.mp4
├── .gitignore
├── requirements.txt
└── README.md
```

A.2 Installation

```
cd Projects/CardGame
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python src/main.py
```

A.3 WSLg Audio Note

On WSLg, Pygame audio requires the PulseAudio client library. The validated environment used:

```
sudo apt update
sudo apt install -y libpulse0 pulseaudio-utils
python src/main.py
```

If audio initialization still fails, the game is designed to continue without sound rather than terminate.

A.4 Controls

- **Continue from start screen:** Any key or mouse click
- **Enter nickname:** Keyboard
- **Select avatar:** Mouse click
- **Adjust bet:** - / + buttons
- **Select turbo:** x1 / x2 / x3 buttons
- **Start round:** START button
- **Choose card:** Mouse click
- **Pause / resume:** SPACE
- **Continue / menu / quit:** On-screen buttons